Foundations of Multilayer perceptron

[Multilayer perceptron]

$$y = \operatorname{sign}(w \cdot x + b)$$

$$w \leftarrow w + \eta(y - \hat{y})x$$

1.0 Content Block

y (v i) = tanh ■ (v i) and y (v i) = (1 + e − v i) − 1 {\displaystyle y(v_{i})=\tanh(v_{i})~~{\textrm {and}}~~y(v_{i})=(1+e^{-v_{i}})^{-1}} . The first is a hyperbolic tangent that ranges from −1 to 1, while the other is the logistic function, which is similar in shape but ranges from 0 to 1. Here y i {\displaystyle y_{i}} is the output of the i {\displaystyle i} th node (neuron) and v i {\displaystyle v_{i}} is the weighted sum of the input connections. Alternative activation functions have been proposed, including the rectifier and softplus functions. More specialized activation functions include radial basis functions (used in radial basis networks, another class of supervised neural network models). In recent developments of deep learning the rectified linear unit (ReLU) is more frequently used as one of the possible ways to overcome the numerical problems related to the sigmoids.

2.0 Content Block

where φ ′ {\displaystyle \phi ^{\prime }} is the derivative of the activation function described above, which itself does not vary. The analysis is more difficult for the change in weights to a hidden node, but it can be shown that the relevant derivative is

3.0 Content Block

− ∂ E (n) ∂ v j (n) = e j (n) φ ′ (v j (n)) {\displaystyle -{\frac {\partial {\mathcal {E}}(n)}{\partial v_{j}(n)}}=e_{j}(n)\phi ^{\prime }(v_{j}(n))}

4.0 Content Block

Δ w j i (n) = − η ∂ E (n) ∂ v j (n) y i (n) {\displaystyle \Delta w_{ji}(n)=-\eta {\frac {\partial {\mathcal {E}}(n)}{\partial v_{j}(n)}}y_{i}(n)}

5.0 Content Block

Activation function If a multilayer perceptron has a linear activation function in all neurons, that is, a linear function that maps the weighted inputs to the output of each neuron, then linear algebra shows that any number of layers can be reduced to a two-layer input-output model. In MLPs some neurons use a nonlinear activation function that was developed to model the frequency of action potentials, or firing, of biological neurons. The two historically common activation functions are both sigmoids, and are described by

6.0 Content Block

The MLP consists of three or more layers (an input and an output layer with one or more hidden layers) of nonlinearly-activating nodes. Since MLPs are fully connected, each node in one layer connects with a

certain weight w i j {\displaystyle w_{ij}} to every node in the following layer.

7.0 Content Block

– ∂ E (n) ∂ v j (n) = φ ′ (v j (n)) ∑ k – ∂ E (n) ∂ v k (n) w k j (n) {\displaystyle -{\frac {\partial {\mathcal {E}}(n)}{\partial v_{j}(n)}}=\phi ^{\prime }(v_{j}(n))\sum _{k}-{\frac {\partial {\mathcal {E}}(n)}{\partial v_{k}(n)}}w_{kj}(n)} . This depends on the change in weights of the k {\displaystyle k} th nodes, which represent the output layer. So to change the hidden layer weights, the output layer weights change according to the derivative of the activation function, and so this algorithm represents a backpropagation of the activation function.

8.0 Content Block

In deep learning, a multilayer perceptron (MLP) is a kind of modern feedforward neural network consisting of fully connected neurons with nonlinear activation functions, organized in layers, notable for being able to distinguish data that is not linearly separable. Modern neural networks are trained using backpropagation and are colloquially referred to as "vanilla" networks. MLPs grew out of an effort to improve on single-layer perceptrons, which could only be applied to linearly separable data. A perceptron traditionally used a Heaviside step function as its nonlinear activation function. However, the backpropagation algorithm requires that modern MLPs use continuous activation functions such as sigmoid or ReLU. Multilayer perceptrons form the basis of deep learning, and are applicable across a vast set of diverse domains.

9.0 Content Block

E (n) = 1 2 ∑ output node j e j 2 (n) {\displaystyle {\mathcal {E}}(n)={\frac {1}{2}}\sum _{{\text{output node }}j}e_{j}^{2}(n)} . Using gradient descent, the change in each weight w i j {\displaystyle w_{ij}} is

10.0 Content Block

Learning Learning occurs in the perceptron by changing connection weights after each piece of data is processed, based on the amount of error in the output compared to the expected result. This is an example of supervised learning, and is carried out through backpropagation, a generalization of the least mean squares algorithm in the linear perceptron. We can represent the degree of error in an output node j {\displaystyle j} in the n {\displaystyle n} th data point (training example) by e j (n) = d j (n) – y j (n) {\displaystyle e_{j}(n)=d_{j}(n)-y_{j}(n)} , where d j (n) {\displaystyle d_{j}(n)} is the desired target value for n {\displaystyle n} th data point at node j {\displaystyle j} , and y j (n) {\displaystyle y_{j}(n)} is the value produced by the perceptron at node j {\displaystyle j} when the n {\displaystyle n} th data point is given as an input. The node weights can then be adjusted based on corrections that minimize the error in the entire output for the n {\displaystyle n} th data point, given by

11.0 Content Block

where y i (n) {\displaystyle y_{i}(n)} is the output of the previous neuron i {\displaystyle i} , and η {\displaystyle \eta } is the learning rate, which is selected to ensure that the weights quickly converge to a response, without oscillations. In the previous expression, ∂ E (n) ∂ v j (n) {\displaystyle {\frac {\partial {\mathcal {E}}(n)}{\partial v_{j}(n)}}} denotes the partial derivate of the error E (n) {\displaystyle {\mathcal {E}}(n)} according to the weighted sum v j (n) {\displaystyle v_{j}(n)} of the input connections of neuron i {\displaystyle i} . The derivative to be calculated depends on the induced local field v j {\displaystyle v_{j}} , which itself varies. It is easy to prove that for an output node this derivative can be simplified to

12.0 Content Block

Timeline In 1943, Warren McCulloch and Walter Pitts proposed the binary artificial neuron as a logical model of biological neural networks. In 1958, Frank Rosenblatt proposed the multilayered perceptron model, consisting of an input layer, a hidden layer with randomized weights that did not learn, and an output layer with learnable connections. In 1962, Rosenblatt published many variants and experiments on

perceptrons in his book Principles of Neurodynamics, including up to 2 trainable layers by "back-propagating errors". However, it was not the backpropagation algorithm, and he did not have a general method for training multiple layers. In 1965, Alexey Grigorevich Ivakhnenko and Valentin Lapa published Group Method of Data Handling. It was one of the first deep learning methods, used to train an eight-layer neural net in 1971. In 1967, Shun'ichi Amari reported the first multilayered neural network trained by stochastic gradient descent, was able to classify non-linearily separable pattern classes. Amari's student Saito conducted the computer experiments, using a five-layered feedforward network with two learning layers. Backpropagation was independently developed multiple times in early 1970s. The earliest published instance was Seppo Linnainmaa's master thesis (1970). Paul Werbos developed it independently in 1971, but had difficulty publishing it until 1982. In 1986, David E. Rumelhart et al. popularized backpropagation. In 2003, interest in backpropagation networks returned due to the successes of deep learning being applied to language modelling by Yoshua Bengio with co-authors. In 2021, a very simple NN architecture combining two deep MLPs with skip connections and layer normalizations was designed and called MLP-Mixer; its realizations featuring 19 to 431 millions of parameters were shown to be comparable to vision transformers of similar size on ImageNet and similar image classification tasks.